

step01_03_5_key_resolutions

October 4, 2026

```
[1]: # # STEP 1.3 - 5 KEY RESOLUTIONS
# A. Resolve duplicate train_no
# B. Identify the 1 unmatched schedule station
# C. Identify the 63 unmatched delay stations
# D. Globally verify (date, train_no, station_no)
# E. Determine the exact grain of FACT_DELAY
#
# Engine: DuckDB (38M rows is handled out-of-core, no chunk-boundary problem).
import json
import duckdb
import pandas as pd
from pathlib import Path

pd.set_option("display.width", 220)
pd.set_option("display.max_columns", 50)
pd.set_option("display.max_rows", 120)

BASE_DIR = Path.cwd().parent
DATA_DIR = BASE_DIR / "data_sets"
RESULT_DIR = BASE_DIR / "results"
RESULT_DIR.mkdir(exist_ok=True)
TMP_DIR = RESULT_DIR / "duckdb_tmp"
TMP_DIR.mkdir(exist_ok=True)

con = duckdb.connect(str(RESULT_DIR / "step01_03_5.duckdb"))
con.execute("SET memory_limit='6GB'") # lower this if your laptop
↳ has < 8 GB RAM
con.execute(f"SET temp_directory='{TMP_DIR.as_posix()}'"")
con.execute("SET preserve_insertion_order=false")

FINDINGS = {} # everything that matters is collected here -> saved as
↳ JSON at the end

def q(sql):
    return con.execute(sql).df()
```

```
def save(df, name):
    df.to_csv(RESULT_DIR / name, index=False)
    print(f"    -> saved results/{name}")
```

```
[2]: # ## 0. Load tables into DuckDB (delay is loaded once and cached in the .duckdb_
    ↪file)
```

```
def load(name, fname, types=None):
    path = (DATA_DIR / fname).as_posix()
    t = f", types={types}" if types else ""
    con.execute(
        f"CREATE OR REPLACE TABLE {name} AS "
        f"SELECT * FROM read_csv('{path}', header=true{t})"
    )

load("train_details",      "train_details.csv",      {"train_no": "BIGINT"})
load("station_full_names", "station_full_names.csv")
load("schedule",           "combined_schedule.csv",  {"train_no": "BIGINT",
    ↪"station_no": "INTEGER"})

already = con.execute(
    "SELECT count(*) FROM information_schema.tables WHERE table_name='delay'"
).fetchone()[0]
if not already:
    print("Loading combined_delay.csv (one-time, a few minutes)...")
    load("delay", "combined_delay.csv",
        {"date": "DATE", "station_no": "INTEGER", "delay": "DOUBLE",
    ↪"train_no": "BIGINT"})

for t in ["train_details", "station_full_names", "schedule", "delay"]:
    print(t, con.execute(f"SELECT count(*) FROM {t}").fetchone()[0])

# Helper tables used repeatedly
con.execute("CREATE OR REPLACE TABLE sched_stations AS SELECT DISTINCT_
    ↪station_name FROM schedule")
con.execute("CREATE OR REPLACE TABLE delay_stations AS SELECT DISTINCT_
    ↪station_name FROM delay")
```

```
train_details 8992
station_full_names 8963
schedule 172112
delay 38428703
```

```
[2]: <_duckdb.DuckDBPyConnection at 0x128cbf6bd30>
```

```

[3]: # ## A. Resolve duplicate train_no
# Question: are the 272 repeated train_no values the *same train* listed under
↳ two type_codes,
# or genuinely different trains that share a number?

print("=== A1. Shape of the duplication ===")

shape = q("""
WITH duplicated AS (
    SELECT
        train_no,
        COUNT(*) AS row_count,
        COUNT(DISTINCT train_name) AS name_count
    FROM train_details
    GROUP BY train_no
    HAVING COUNT(*) > 1
)
SELECT
    COUNT(*) AS dup_train_nos,
    SUM(CASE WHEN row_count > 2 THEN 1 ELSE 0 END) AS with_more_than_2_rows,
    SUM(CASE WHEN name_count > 1 THEN 1 ELSE 0 END) AS with_different_names
FROM duplicated
""")

print(shape.to_string(index=False))

print("\n=== A2. Which type_code combinations occur among duplicates? ===")
combos = q("""
SELECT combo, count(*) AS n_train_no
FROM (SELECT train_no, string_agg(DISTINCT type_code, ' + ' ORDER BY type_code)
↳ AS combo
    FROM train_details GROUP BY train_no HAVING count(*) > 1)
GROUP BY combo ORDER BY n_train_no DESC
""")

print(combos.to_string(index=False))
save(combos, "step01_03_5_A_duplicate_type_combos.csv")

print("\n=== A3. Are duplicated train numbers actually used in schedule / delay?
↳ ===")
usage = q("""
WITH d AS (SELECT train_no FROM train_details GROUP BY train_no HAVING count(*)
↳ > 1)
SELECT count(*) AS dup_train_nos,
    sum(CASE WHEN train_no IN (SELECT train_no FROM schedule) THEN 1 ELSE 0
↳ END) AS in_schedule,
    sum(CASE WHEN train_no IN (SELECT train_no FROM delay) THEN 1 ELSE 0
↳ END) AS in_delay
""")

```

```

FROM d
"""
print(usage.to_string(index=False))

# Resolution rule (hypothesis, TESTED by A2):
#   PRM-TRAINS behaves like an overlay tag ("premium/special") on top of a base
#   ↳ service type.
#   -> keep the base type (EXP / SF / SHT / RAJ / ...) as primary_type_code,
#       keep the PRM info as a boolean flag. If a group has >1 distinct non-PRM
#       ↳ types -> 'REVIEW'.
con.execute("""
CREATE OR REPLACE TABLE dim_train_resolved AS
SELECT train_no,
       min(train_name) AS train_name,
       count(DISTINCT train_name) AS n_names_raw,
       CASE
         WHEN count(DISTINCT type_code) FILTER (WHERE type_code <>
↳ 'PRM-TRAINS') = 1
           THEN min(type_code) FILTER (WHERE type_code <>
↳ 'PRM-TRAINS')
         WHEN count(DISTINCT type_code) FILTER (WHERE type_code <>
↳ 'PRM-TRAINS') = 0
           THEN 'PRM-TRAINS'
         ELSE 'REVIEW'
       END AS primary_type_code,
       bool_or(type_code = 'PRM-TRAINS') AS is_premium_special,
       string_agg(DISTINCT type_code, '|' ORDER BY type_code) AS raw_type_codes,
       count(*) AS raw_row_count
FROM train_details
GROUP BY train_no
""")

res = q("""
SELECT
  COUNT(*) AS resolved_row_count,
  COUNT(DISTINCT train_no) AS unique_train_no,
  SUM(CASE WHEN primary_type_code = 'REVIEW' THEN 1 ELSE 0 END)
    AS needs_manual_review,
  SUM(CASE WHEN n_names_raw > 1 THEN 1 ELSE 0 END)
    AS name_conflicts,
  SUM(CASE WHEN is_premium_special THEN 1 ELSE 0 END)
    AS premium_flagged
FROM dim_train_resolved
""")
print("\n=== A4. Resolved train dimension ===")
print(res.to_string(index=False))

```

```

print("\nprimary_type_code distribution after resolution:")
print(q("SELECT primary_type_code, count(*) AS n FROM dim_train_resolved GROUP_
↳BY 1 ORDER BY n DESC").to_string(index=False))

review = q("SELECT * FROM dim_train_resolved WHERE primary_type_code='REVIEW'_
↳OR n_names_raw>1")
if len(review):
    print("\n!! Rows needing manual decision:")
    print(review.to_string(index=False))
    save(review, "step01_03_5_A_train_manual_review.csv")

save(q("SELECT * FROM dim_train_resolved ORDER BY train_no"),_
↳"dim_train_resolved.csv")

FINDINGS["A"] = {
    "raw_rows": 8992,
    "dup_train_nos": int(shape.dup_train_nos[0]),
    "dup_with_different_names": int(shape.with_different_names[0]),
    "resolved_rows": int(res.resolved_row_count[0]),
    "resolved_unique_train_no": int(res.unique_train_no[0]),
    "needs_manual_review": int(res.needs_manual_review[0]),
    "rule": "primary_type_code = the single non-PRM type; PRM kept as_
↳is_premium_special flag",
}
assert res.resolved_row_count[0] == res.unique_train_no[0], "train_no still not_
↳unique after resolution!"

```

=== A1. Shape of the duplication ===

dup_train_nos	with_more_than_2_rows	with_different_names
272	0.0	0.0

=== A2. Which type_code combinations occur among duplicates? ===

combo	n_train_no
PRM-TRAINS + T18-TRAINS	194
EXP-TRAINS + PRM-TRAINS	40
PASS-TRAINS + PRM-TRAINS	17
PRM-TRAINS + SHT-TRAINS	14
PASS-TRAINS + T18-TRAINS	4
PRM-TRAINS + RAJ-TRAINS	3

-> saved results/step01_03_5_A_duplicate_type_combos.csv

=== A3. Are duplicated train numbers actually used in schedule / delay? ===

dup_train_nos	in_schedule	in_delay
272	264.0	248.0

=== A4. Resolved train dimension ===

resolved_row_count	unique_train_no	needs_manual_review	name_conflicts
--------------------	-----------------	---------------------	----------------

```

premium_flagged
            8720            8720            4.0            0.0
448.0

```

primary_type_code distribution after resolution:

```

primary_type_code    n
PASS-TRAINS    4534
EXP-TRAINS    2255
SF-TRAINS    1382
T18-TRAINS    194
PRM-TRAINS    180
SHT-TRAINS    64
GRB-TRAINS    54
RAJ-TRAINS    53
REVIEW        4

```

!! Rows needing manual decision:

```

train_no      train_name  n_names_raw primary_type_code
is_premium_special  raw_type_codes  raw_row_count
94801 NAMO BHARAT RAPID RAIL      1      REVIEW
False PASS-TRAINS|T18-TRAINS      2
94804 NAMO BHARAT RAPID RAIL      1      REVIEW
False PASS-TRAINS|T18-TRAINS      2
94803 NAMO BHARAT RAPID RAIL      1      REVIEW
False PASS-TRAINS|T18-TRAINS      2
94802 NAMO BHARAT RAPID RAIL      1      REVIEW
False PASS-TRAINS|T18-TRAINS      2
-> saved results/step01_03_5_A_train_manual_review.csv
-> saved results/dim_train_resolved.csv

```

```

[4]: # A5: Finalize Manual Train Type Resolution
print("\n=== A5. Finalize Manual Train Type Resolution ===")

# Manual Review Completed: Four specific 'NAMO BHARAT RAPID RAIL' rows required,
  ↳ manual intervention.
# These are manually updated to the 'T18-TRAINS' primary type code.
con.execute("""
UPDATE dim_train_resolved
SET primary_type_code = 'T18-TRAINS'
WHERE train_name = 'NAMO BHARAT RAPID RAIL'
  AND primary_type_code = 'REVIEW'
""")

res_review = q("SELECT count(*) as remaining_reviews FROM dim_train_resolved,
  ↳ WHERE primary_type_code = 'REVIEW'")
print(f"Remaining reviews needed: {res_review['remaining_reviews'][0]}")

```

```
FINDINGS["A"]["remaining_reviews_after_manual"] =
    ↪int(res_review['remaining_reviews'][0])

# Finalize the 8,720-row train dimension without dropping any valid records
save(q("SELECT * FROM dim_train_resolved ORDER BY train_no"),
    ↪"dim_train_resolved.csv")
```

=== A5. Finalize Manual Train Type Resolution ===

Remaining reviews needed: 0

-> saved results/dim_train_resolved.csv

```
[5]: # === B & C. Investigate Unmatched Stations ===
print("\n=== B & C. Unmatched Stations in Schedule and Delay ===")
# Isolate the missing stations using our cached helper tables
missing_sched = q("""
    SELECT station_name FROM sched_stations
    WHERE station_name NOT IN (SELECT station_name FROM station_full_names)
""")
missing_delay = q("""
    SELECT station_name FROM delay_stations
    WHERE station_name NOT IN (SELECT station_name FROM station_full_names)
""")

print(f"Unmatched in schedule ({len(missing_sched)}): \n{missing_sched.
    ↪to_string(index=False)}")
print(f"\nUnmatched in delay ({len(missing_delay)}): \n{missing_delay.head().
    ↪to_string(index=False)}\n... (truncated)")

# Resolve referential integrity by appending missing codes with placeholder
    ↪metadata
con.execute("""
    CREATE OR REPLACE TABLE dim_station_resolved AS
    SELECT station_name, station_full_name, station_zone, station_address
    FROM station_full_names
    UNION ALL
    SELECT station_name, 'UNKNOWN' AS station_full_name, 'UNKNOWN' AS
    ↪station_zone, 'UNKNOWN' AS station_address
    FROM (
        SELECT station_name FROM sched_stations
        UNION
        SELECT station_name FROM delay_stations
    )
    WHERE station_name NOT IN (SELECT station_name FROM station_full_names)
""")

res_stations = q("SELECT count(*) as total_stations FROM dim_station_resolved")
```

```

save(q("SELECT * FROM dim_station_resolved ORDER BY station_name"),
    ↪ "dim_station_resolved.csv")

FINDINGS["B_C"] = {
    "missing_sched_count": len(missing_sched),
    "missing_delay_count": len(missing_delay),
    "final_station_dim_count": int(res_stations['total_stations'][0])
}

```

=== B & C. Unmatched Stations in Schedule and Delay ===

Unmatched in schedule (1):

```

station_name
    CLDB

```

Unmatched in delay (63):

```

station_name
    KCVL
    JFD
    OKD
    INCI
    AE

```

... (truncated)

→ saved results/dim_station_resolved.csv

```

[6]: # === D. Globally verify (date + train_no + station_no) ===
print("\n=== D. Global Uniqueness Verification ===")
print("Executing out-of-core global duplicate check on 38M rows...")
duplicates_check = q("""
    SELECT date, train_no, station_no, count(*) as count
    FROM delay
    GROUP BY date, train_no, station_no
    HAVING count(*) > 1
""")

n_dups = len(duplicates_check)
print(f"Duplicate combinations found: {n_dups}")
FINDINGS["D"] = {"duplicate_grain_combinations": n_dups}

# === E. Determine the exact grain of FACT_DELAY ===
print("\n=== E. Determine Final Grain ===")
if n_dups == 0:
    print("SUCCESS: The combination (date, train_no, station_no) is globally
    ↪ unique.")
    print("Grain for FACT_DELAY locked in: 1 row = 1 train's delay at 1
    ↪ sequential station on 1 calendar date.")

```

```

    FINDINGS["E"] = {"fact_grain": "(date, train_no, station_no)",
↳ "is_globally_unique": True}
else:
    print(f"WARNING: {n_dups} duplicate combinations exist. ETL will require an
↳ aggregation rule (e.g., MAX(delay)).")
    FINDINGS["E"] = {"fact_grain": "(date, train_no, station_no)",
↳ "is_globally_unique": False}

# Save accumulated findings to disk
with open(RESULT_DIR / "step01_03_5_FINDINGS.json", "w") as f:
    json.dump(FINDINGS, f, indent=4)
print("\n -> saved results/step01_03_5_FINDINGS.json")

```

=== D. Global Uniqueness Verification ===

Executing out-of-core global duplicate check on 38M rows...

```

FloatProgress(value=0.0, layout=Layout(width='auto'),
↳ style=ProgressStyle(bar_color='black'))

```

Duplicate combinations found: 0

=== E. Determine Final Grain ===

SUCCESS: The combination (date, train_no, station_no) is globally unique.

Grain for FACT_DELAY locked in: 1 row = 1 train's delay at 1 sequential station on 1 calendar date.

-> saved results/step01_03_5_FINDINGS.json

[]: